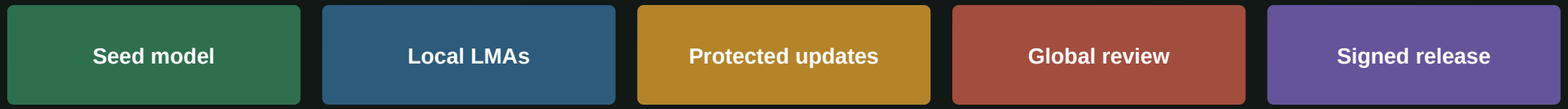
FEDERATED HEALTHCARE USE CASE

Dagents for Federated Stroke Triage

Send a signed stroke-triage model to hospitals, train and evaluate locally, aggregate protected updates, and release only after cross-site review.



Pradyun Devarakonda

THE PROBLEM

Pooling raw hospital records is not the only way to learn.

The system must handle data boundaries, site differences, update privacy, and safe operations—not just model accuracy.

Data cannot simply move

Privacy, contracts, sovereignty, and breach risk make a central patient-data copy a poor default.

Hospitals do not match

Populations, units, systems, labels, and workflows differ. One global setting may not fit every site.

Updates can still leak

Weights and gradients are not automatically anonymous. Aggregation and privacy controls still matter.

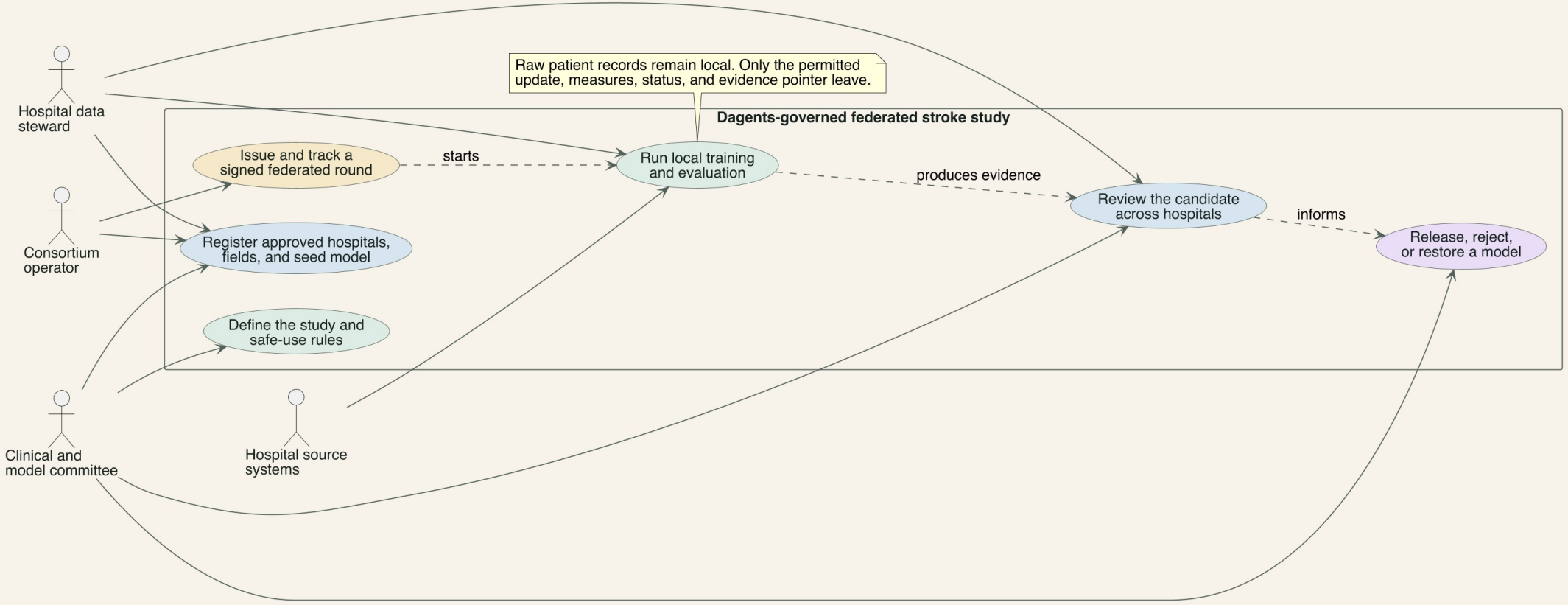
Operations decide safety

Identity, code approval, retries, versions, evidence, canaries, and rollback determine whether learning is controllable.

USE CASE

The system has clear actors, permissions, and outcomes.

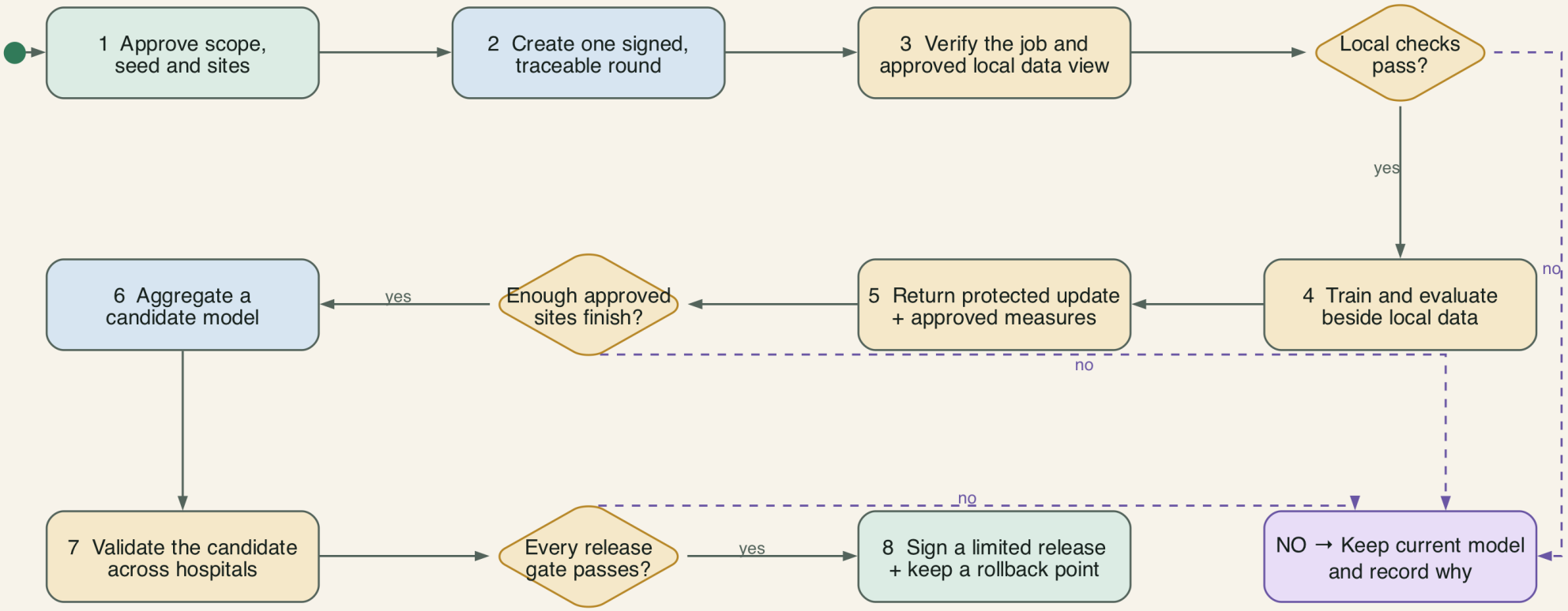
The hospital keeps control of local data. Dagents governs the study and release. The federated runtime performs the distributed job.



FUNCTIONAL FLOW

Every round follows one visible path with explicit stop points.

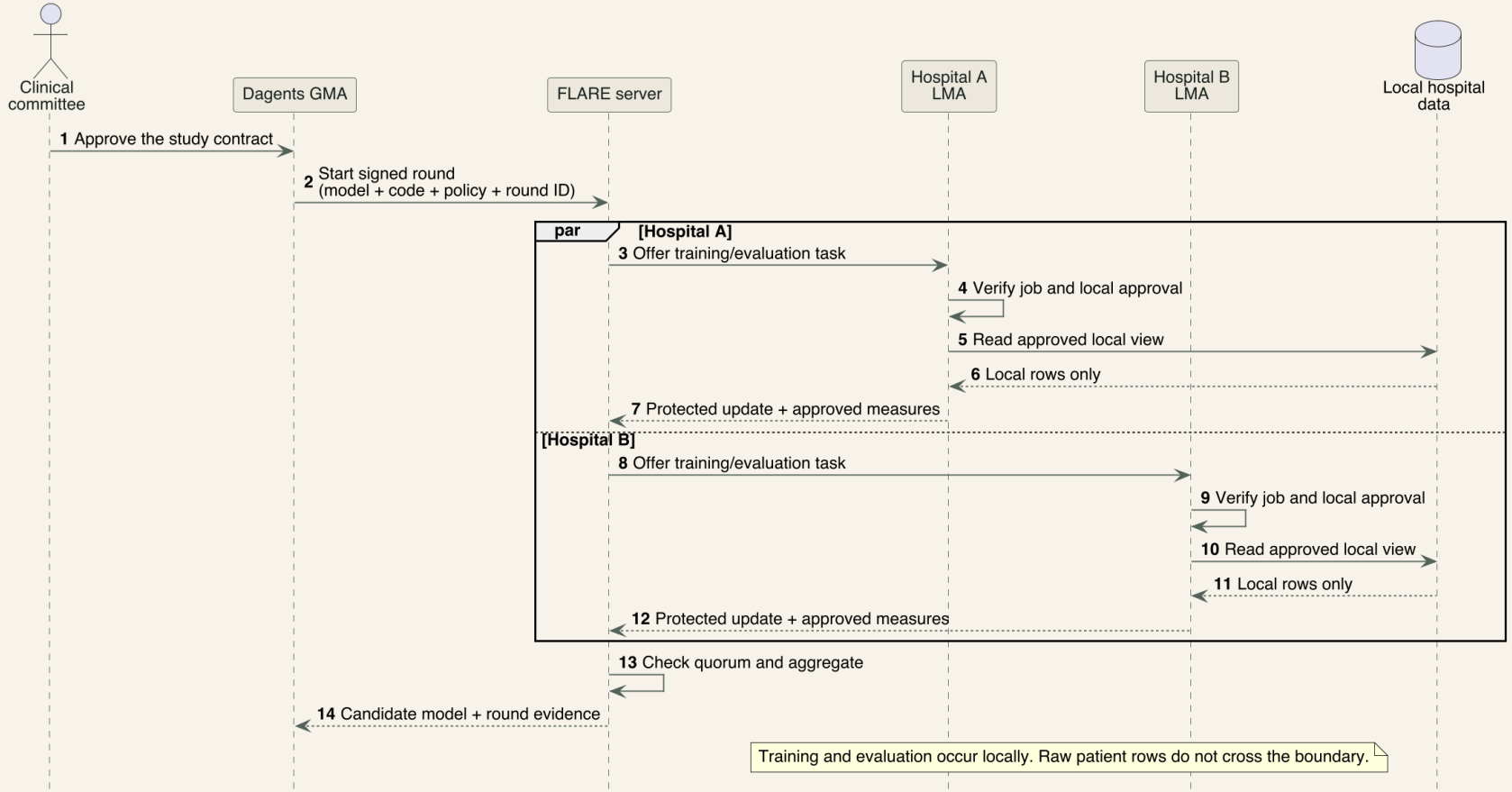
A failed local check, missing quorum, or failed release gate keeps the current approved model in service.



SEQUENCE 1 OF 2

A federated round creates a candidate without pooling patient records.

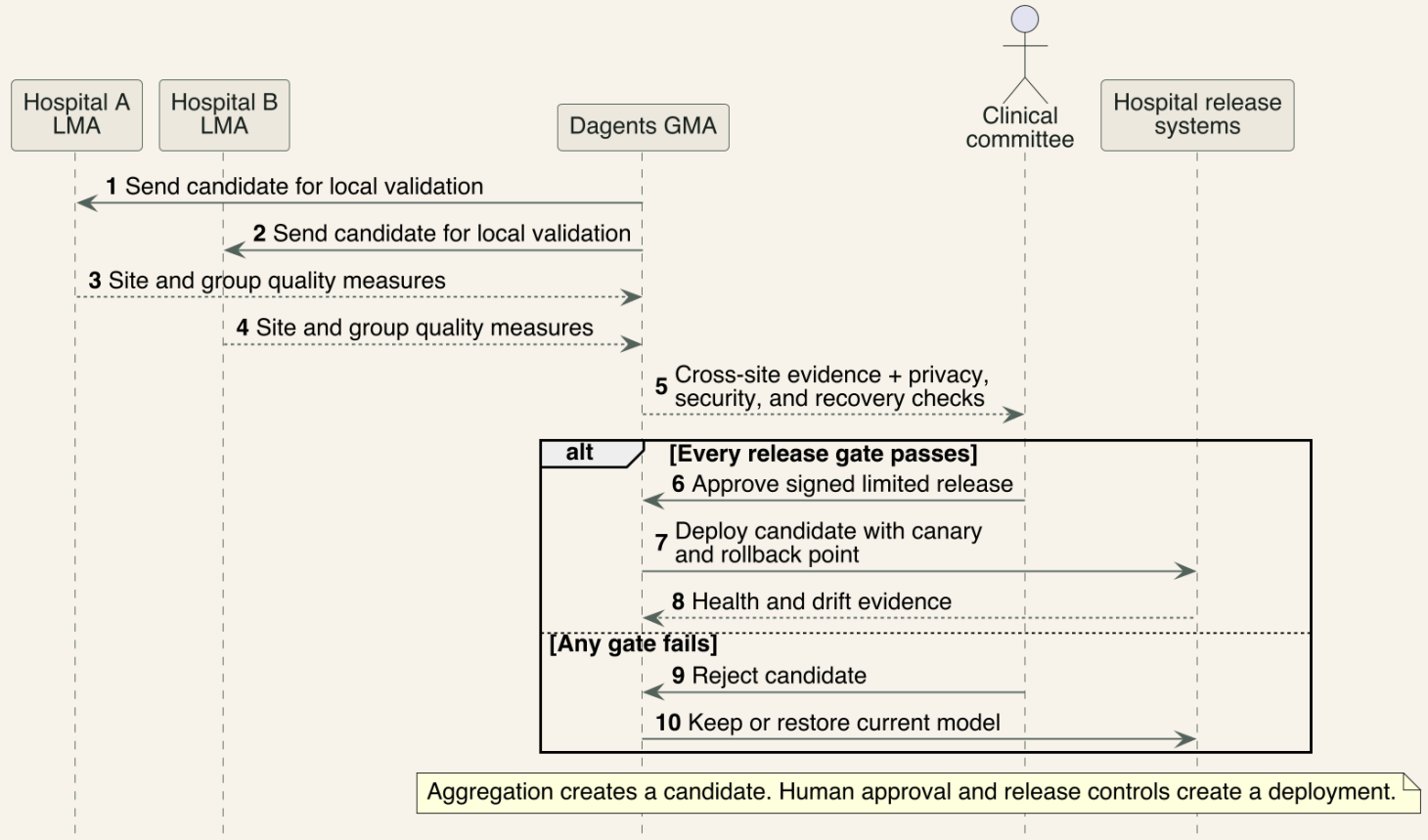
Dagents approves and tracks the round. FLARE coordinates the job. Each LMA checks and runs work beside local data.



SEQUENCE 2 OF 2

A candidate becomes a release only after cross-site review.

Hospitals return local quality measures. The committee either approves a limited release or keeps the current model.



USE-CASE BOUNDARY

Start narrow enough to prove the whole system.

Illustrative research pilot: five hospitals, emergency and stroke pathways, past-scan testing followed by silent workflow evaluation.

One clinical question

Can a model help prioritize suspected-stroke scans for faster specialist review? A clinical committee defines inputs, output, exclusions, and safe use.

Approved seed

Start from public, de-identified, consented, synthetic, or otherwise permitted data—not a central copy of hospital records.

No automatic care

Past scans and silent mode first. No diagnosis, treatment, or queue removal without a separate approved study.

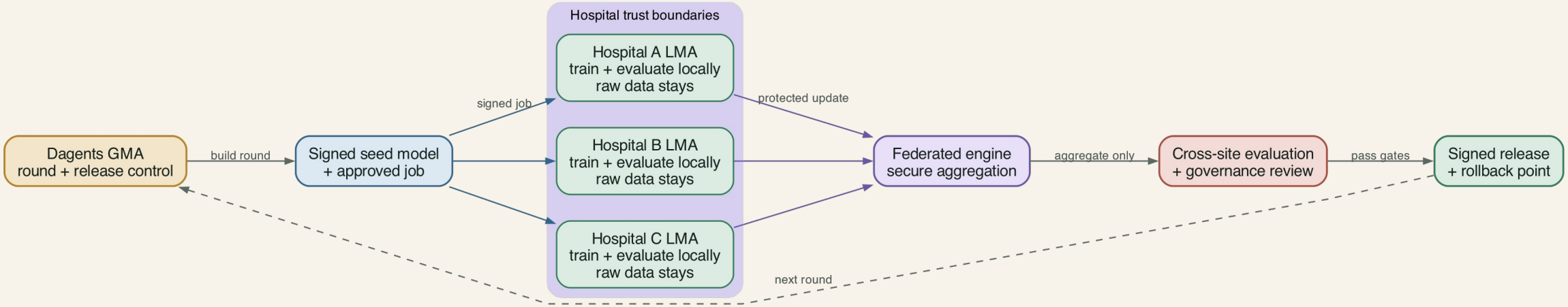
One condition

Later cases could cover heart attack, sudden kidney injury, breathing decline, or diabetic emergency. Each needs its own model, workflow, owner, and safety evidence.

FEDERATED ARCHITECTURE

Dagents controls the round; a federated engine aggregates.

Signed jobs go out, protected updates come back, and raw records plus patient-level traces stay at each hospital.



RELEASE LIFECYCLE

Treat every model version as a recoverable release.

Federated training creates candidates; governance creates releases.

BASELINE

Record the seed model, code, features, dependencies, intended use, approved sites, and safe fallback.

INCREMENT

Create a round manifest; collect bounded updates and approved aggregate results.

VERIFY

Re-evaluate at every site; check privacy, performance, subgroup, integrity, and operational gates.

RELEASE OR RESTORE

Sign a candidate for a small rollout—or keep and restore the last known-good model and settings.

No candidate silently replaces the approved model.

LOCAL AGENT (LMA)

The LMA is the hospital’s enforcement point.

It accepts only approved jobs, touches only approved fields, and keeps the detailed trace under hospital control.

VERIFY JOB

Check signer, code hash, dependencies, condition, allowed fields, privacy settings, and local approval. The hospital may reject.

PREPARE LOCALLY

Build the approved feature view; validate units, time windows, missingness, cohort, and label availability.

TRAIN + EVALUATE

Run an isolated workload, calculate local measures, and clip or bound the site contribution.

PUBLISH MINIMUM

Send only the protected update, allowed measures, status, versions, and a local evidence pointer.

Detailed records, features, and patient-level predictions never belong in the GMA result.

GLOBAL AGENT (GMA)

The GMA controls participation, rounds, and releases.

It coordinates the federation; it does not become a raw patient database.

REGISTER

Track each hospital, LMA, approved use, capability, schema version, certificate, and sharing rule.

PLAN ROUND

Choose participants, minimum threshold, seed version, privacy profile, evaluation plan, and stop conditions.

COORDINATE

Send signed jobs; track acknowledgements, timeouts, dropouts, retries, cancellation, and aggregate status.

RELEASE + ROLLBACK

Collect cross-site evidence, sign a canary release, watch health, or restore the known-good version.

When secure aggregation is required, the GMA should not read any hospital’s individual update.

METHOD BOUNDARIES

Four ‘federated’ ideas solve different problems.

Use them together, but do not treat the names as interchangeable.

Method	What it does	Dagents role / important limit
Federated learning	Trains a shared model from updates computed at each hospital	Use a specialist FL engine; Dagents coordinates jobs, evidence, releases, and rollback
Federated analytics	Computes local counts, statistics, and evaluation; reveals approved aggregates	Best first phase; small groups and repeated queries can still reveal sensitive facts
Federated database	Offers one query layer across remote sources and can push work to the source	Useful inside a hospital; unrestricted cross-site patient queries are out of scope
Central warehouse	Copies data into one place for central processing and analytics	Useful for some needs, but not a required dependency for this time-sensitive path
Backup patterns	Protect versions with baselines, increments, catalogs, checks, and recovery	Operations model only; backup software does not train or validate a clinical model

NVIDIA FLARE CASE STUDY

EXAM was a federated COVID-19 outcome study.

‘Electronic Medical Record Chest X-ray AI Model'. It predicted oxygen needs 24 and 72 hours after emergency-department presentation.

20 institutions, 16,148 cases

Hospitals and research sites across several countries trained and tested one shared research model.

What the model used

Each site used its first emergency-department chest X-ray plus 19 approved record features such as vital signs, laboratory values, and age.

How federation worked

Sites trained locally. A central server combined model updates. Raw site datasets were not copied into one repository.

What the paper reported

Average AUC was above 0.92. For the 24-hour task, the federated model improved average AUC by 16% and average generalizability by 38% versus site-only models.

Nature Medicine, 2021. COVID-19 research evidence, not a stroke result or regulatory approval.

WHAT DAGENTS SHOULD LEARN

The lesson is the operating model, not the brand.

A real healthcare federation needs repeatable experiments, site workers, evaluation, privacy controls, and release governance.

Many traceable rounds

The EXAM study ran many experiments. Dagents needs durable round state, inputs, outputs, versions, and reasons.

Local site capability

Each hospital needs approved compute, data preparation, training, evaluation, and evidence storage.

Heterogeneity stays

Shared features do not erase site differences. Every candidate needs site-level and group-level evaluation.

Code + privacy approval

Hospitals should verify job code and data access; secure aggregation and privacy controls remain separate work.

Integrate NVIDIA FLARE or an equivalent runtime; keep Dagents as the reusable governance and orchestration layer / 14

ENTERPRISE BACKUP LESSONS

Backup patterns make model releases safer.

Borrow central policy, local workers, integrity checks, and recovery—not the idea that an update is automatically trustworthy.

Baseline + increments

A full backup plus changed blocks maps to a signed seed package plus bounded round updates.

Catalog + lineage

A backup catalog maps to model, code, feature-contract, round, participant, candidate, and release history.

Integrity checks

Checksums, signatures, reproducible evaluation, and locked releases detect corruption or silent change.

Restore drills

A rollback plan is incomplete until teams regularly prove that the known-good model can be restored.

DAGENTS MAPPING

Dagents supplies the control plane around specialist federation.

Five framework capabilities compose the use case; no single service is the whole solution.

LMA near the data

Verify the job, build local features, train or evaluate, keep detailed evidence, and publish the minimum result.

Pipeline plans

Define repeatable readiness, training, evaluation, packaging, approval, rollout, and recovery steps.

Federated engine

Run FedAvg or another approved algorithm, secure aggregation, privacy mechanisms, and dropout handling.

Model registry

Store signed seeds, candidates, releases, code digests, feature contracts, evidence pointers, and rollback targets.

GMA + core

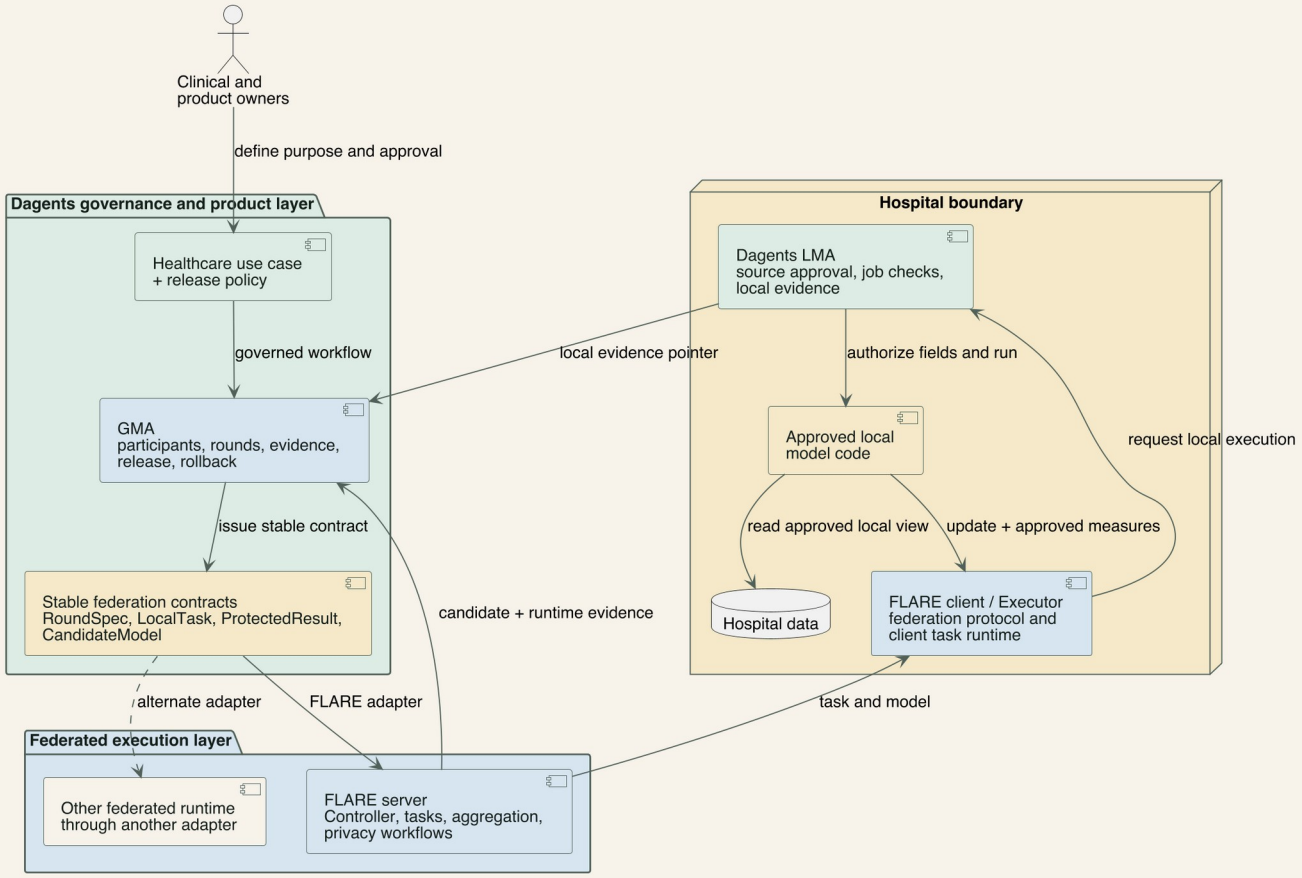
Register sites, plan rounds, compile workloads, coordinate release, monitor health, pause, and restore.

Controls across every step: identity, least privilege, minimum messages, signatures, retention, evidence, and human approval.

DAGENTS + FLARE BOUNDARY

Dagents can sit above FLARE and beside it at each hospital.

Above: Dagents keeps stable healthcare contracts and release policy. Beside: the LMA gates local access while the FLARE client runs the federation protocol.



Dagents and FLARE work together at different layers.

Dagents keeps one healthcare contract. Adapters translate it to FLARE or another approved runtime.

Question	NVIDIA FLARE	Dagents
Primary job?	Run the federation protocol: controller, tasks, clients, aggregation, and supported privacy workflows	Define the use case, eligible sites, signed round contract, evidence, release, and rollback
Inside a hospital?	Client API or Executor connects the protocol to local model code	LMA approves the job and source view, then keeps detailed local evidence
Across hospitals?	Controller dispatches tasks, collects contributions, and builds a candidate	GMA sets quorum and stop rules, records the round, and governs the result contract
What is produced?	Candidate model plus runtime status and validation support	Governed candidate and evidence package; human approval remains separate
Can the runtime change?	Runtime-specific job, controller, and client APIs	Stable contracts can map to FLARE now or another approved runtime later

Design rule: do not rebuild FLARE inside GMA, and do not ask FLARE to replace Dagents governance. / 18

COMPLIANCE + SAFETY

Less data movement does not equal compliance.

Technical controls help, but hospitals and the consortium still own policy, operations, clinical evidence, and accountable decisions.

HIPAA

Keeping records local can reduce copies. Teams still need approved purpose, minimum-necessary decisions, access control, encryption, BAAs, retention, audit, risk analysis, and incident response.

SOC 2

Dagents can produce health, change, and recovery evidence. The organization still needs control owners, access reviews, change management, availability targets, vendor risk work, and an independent examination.

CLINICAL AI SAFETY

Each stroke-triage model needs a defined intended use, site and group validation, usability testing, clinician authority, monitoring, rollback, and regulatory analysis before care use.

ROUND SAFETY GATES

Four gates must pass before a round changes the model.

Any failure cancels the candidate, records the reason, and keeps the current release in service.

1. SCOPE + SITE

Is the hospital, condition, patient group, seed version, feature contract, and purpose approved?

2. CODE + DATA

Did the hospital verify the signer and code hash? Are units, cohort, labels, freshness, and quality acceptable?

3. PRIVACY

Are contributions bounded, participant threshold met, egress allowed, and aggregation/privacy settings correct?

4. RELEASE

Did every site evaluation, subgroup, security, clinical, canary, and rollback check pass?

A successful aggregation is only a candidate—not permission to deploy.

A published way to decide who may see what.

GRAILS keeps the ethics rules out of the normal application code, so a policy change does not become a code change.

Two parts

The Rails work out what protection a request needs. The Guard applies it. Deciding and enforcing stay separate.

Three questions

How sensitive is the data? How far do we trust who is asking? How much are they asking for?

Added, not rebuilt

It plugs into software that already exists. The stroke pathway is not redesigned to add ethics to it.

Wider than privacy

Privacy is the paper's worked example. The same shape extends to fairness, transparency, and security.

Access rules based on job titles alone cannot express “this person, this field, this purpose, today”. GRAILS can.

The three dials, using stroke data.

The same request can get a different answer, depending on where the three dials sit.

HOW SENSITIVE

A ward name is low. The time symptoms started is medium. The brain scan and the final diagnosis are high.

HOW TRUSTED

A treating clinician on the stroke pathway, an internal audit reviewer, and an outside researcher are not the same requester.

HOW MUCH

One value for one patient, one patient's whole record, one field across every stroke case, or the entire table.

High sensitivity, asked for by low trust, across a whole column is the case that must never quietly default to allow.

GRAILS + STROKE

The same field, three different answers.

What a requester gets back depends on who they are, not only on what they asked for.

Stroke field	Treating clinician	Audit reviewer	Outside researcher
When symptoms started	Exact time	Rounded to the hour	Time band only
Brain scan image	Full image	Report text, not the image	Not released
Stroke severity score	Exact score	Grouped into bands	Bands, small groups hidden
Final diagnosis	Full detail	Grouped category	Counts only, minimum group size

None of this is an if-statement inside the application. The row is looked up from the three dials.

GRAILS splits the same way Dagents already splits.

The Rails are planning. The Guard is runtime. That line is the one Dagents is built on.

Rails = planning

Work out the protection plan. Pure, typed, repeatable, and no input or output. This belongs in the OCaml layer, beside the dataset and model planners.

Guard = runtime

Apply the plan to real data and record what was decided. This belongs in the LMA and GMA, where the side effects already are.

GRAILS part	Where it goes in Dagents	Why there
Restriction Planner	OCaml planner module	The paper compares it to a database query planner. Dagents already has that layer.
Ethical Guard	LMA and GMA request path	Enforcement needs real data, a real requester, and an audit log.
Rules and regulations	Configuration, not code	Hospital policy changes far more often than the framework does.

The guard stands at the four gates you already have.

GRAILS does not add a fifth gate. It makes the existing ones concrete and reviewable.

1. SCOPE + SITE

The trust score decides whether this hospital, this study, and this requester may run at all.

2. CODE + DATA

Only fields on the stroke feature contract can be read. Anything else is refused before a scan is ever opened.

3. PRIVACY

The restriction plan sets the limit on the update and the counts before they leave the hospital network.

4. RELEASE

The GMA checks the plan was followed and the evidence exists before it signs anything as released.

Every decision writes down what it decided, why, and which round it belonged to.

GRAILS protects rows. A stroke round sends updates.

This is the one place the framework has to be extended, and it is the contribution this project can make.

What it covers today

One value, one row, one column, or a whole table. All of them are data you can point at and read.

What a stroke round adds

A model update is not a row, but it still carries patient signal. It has to become a fifth thing the rules cover.

Sensitivity becomes

How much stroke-patient signal an update can carry out of the hospital.

Trust becomes

The receiving site and the coordinator, not only a named person.

Action becomes

Limit the contribution, require enough sites, add noise, or refuse to send.

This is a real extension of the published framework, not a restatement of it.

CHOICE

Why Dagents over a framework or SaaS alone?

Best fit: Dagents supplies reusable control and release governance around a specialist federated-learning engine.

Option	Advantages	Tradeoffs
Dagents + FL engine	Source-local LMA boundary; reusable GMA orchestration; portable contracts; evidence and rollback in one design	More integration and operational ownership; privacy, identity, and clinical controls still need production work
FL framework alone	Mature algorithms, simulation, aggregation, and privacy extensions	Needs a surrounding control plane for orchestration, evidence, releases, and product integration
Managed FL SaaS	Fast onboarding, hosted services, and vendor support	Lock-in, recurring cost, deployment constraints, and less control over evidence or recovery
Central warehouse + ML	Familiar central analytics and model tools	Large and delayed data flows, duplicated sensitive records, and a larger governance boundary
One-off custom build	Can match one hospital network exactly	Slow to reuse and maintain; likely to create incompatible contracts for the next condition

Dagents wins on control, portability, and recoverability—not on lowest initial effort / 27

PILOT ROADMAP

Prove federation in stages.

Start with federated analytics; enable training only after the data, privacy, and recovery paths work.

Phase	What happens	Proof needed
1. Define	Approve the stroke-triage question, patient group, scan type, outcome, sources, site owners, and stop conditions	Signed clinical and data contract; named owners; no automatic care action
2. Federated analytics	Compute local counts, missingness, units, labels, and seed-model measures; reveal approved aggregates only	Same meaning at every site; minimum-message path; privacy threshold and audit trail
3. Simulate	Exercise complete rounds with approved test data, including site rejection, dropout, retry, bad update, and rollback	Deterministic recovery, signatures, round state, and no individual-update exposure
4. Retrospective train	Train on historical local data; evaluate every candidate locally and compare with local-only and current baselines	Cross-site and subgroup evidence; privacy report; candidate rejected when any gate fails
5. Silent live	Run the approved model on current data without displaying warnings to clinicians	Reliability, drift, alert volume, security, recovery, and separate clinical approval for any next stage

SUCCESS MEASURES

Success means quality, privacy, and recovery.

No single accuracy number proves that the federated system is safe or useful.

Cross-site model evidence

Calibration and discrimination at every hospital; approved group results; comparison with local-only and current baselines.

Minimal data movement

Zero raw patient rows in the GMA path; measured message size; blocked fields; cohort and privacy-threshold compliance.

Reliable rounds

Completion, timeout, dropout, retry, cancellation, signature verification, and time to stop a bad round.

Proof of recovery

Complete seed-to-release lineage; immutable approved versions; successful scheduled rollback and evidence-recovery drills.

RECOMMENDATION

Start with federated analytics; earn the right to train.

Use Dagents as the governed orchestration and release layer around a specialist federated-learning engine.

- Define one stroke-triage research use, feature contract, participant policy, site owner, and safe fallback.
- Run data-readiness and seed-model evaluation locally; share only protected aggregate measures.
- Integrate NVIDIA FLARE or an equivalent engine behind Dagents round and result contracts.
- Require local code approval, secure aggregation where needed, signed releases, and complete lineage.
- Practice cancellation and rollback before retrospective training—and again before any clinician-facing study.

Federated-system terms

The terms describe different jobs; none is automatic proof of privacy, security, safety, or compliance.

Term	Plain meaning	Why it matters here
Federated learning	Hospitals train locally and send protected model updates instead of records	Improves one shared model; requires a specialist engine and privacy controls
Federated analytics	Hospitals calculate approved measures locally and reveal combined results	The safest first cross-hospital workload for readiness and evaluation
Federated database	One query layer can reach several remote data sources	Useful for local feature views; can still expose or move detailed results
Secure aggregation	The coordinator learns a combined update without reading each site’s update	Reduces exposure; does not solve poisoning, memorization, or governance alone
Restore point / rollback	A locked known-good release / return to that release after a problem	Backup-inspired control that makes model changes reversible

FEDERATED AI APPENDIX

Federated AI

A self-contained guide to NVIDIA FLARE, federated rounds, FedAvg, privacy layers, deployment modes, and the Dagents boundary.

Big picture | NVIDIA FLARE | One round | Privacy layers | Dagents fit

Pradyun Devarakonda

APPENDIX · BIG PICTURE

Federated AI is an umbrella, not one algorithm.

The common idea is to move approved computation or small results instead of pooling every raw record.

Term	Plain meaning	What crosses the boundary
Federated AI	Broad family of distributed AI and analytics methods	Tasks, models, updates, or aggregates
Federated learning	Sites improve one shared model by training locally	Protected model updates and approved measures
Federated analytics	Sites calculate approved statistics locally	Counts, statistics, and evaluation summaries
Federated database	One query layer reaches several remote sources	Queries and results—possibly detailed rows
Privacy techniques	Extra protection for messages and released results	Masked values, encryption, limits, and noise

Use the precise term: federation is a design family, not an automatic privacy guarantee / A2

APPENDIX · NVIDIA FLARE

NVIDIA FLARE is a runtime for multi-site computing.

It is an open-source Python SDK for federated jobs—not a hospital data lake, stroke model, or compliance certificate.

Job definition	Server workflow	Client runtime	Algorithms	Security + operations
Packages the model, client code, sites, rounds, aggregation method, and validation plan.	A controller creates tasks, selects clients, tracks rounds, and processes returned results.	An executor or Client API connects FLARE to local training or evaluation beside the data.	Supports FedAvg, FedProx, FedOpt, other workflows, and cross-site validation patterns.	Provisioning, mTLS, site policies, filters, audit, simulation, POC, and production modes.

FLARE runs the specialist distributed workflow; Dagents still governs purpose, evidence, release, and recovery.

APPENDIX · FLARE VOCABULARY

A FLARE job has four names worth knowing.

Read them as a simple request-and-return system; the lower-level software details can wait.

JOB / RECIPE

The packaged plan: model, training code, clients, rounds, aggregation, and validation.

CONTROLLER

Server-side logic that creates tasks, chooses clients, and decides what happens next.

TASK

One unit of client work, such as train this model or evaluate this candidate.

EXECUTOR / CLIENT API

The client-side bridge that calls local code and returns only the permitted result.

The task payload may use FLModel or the lower-level Shareable type; neither should contain the hospital's patient database.

APPENDIX OVERVIEW · ONE ROUND

One round is a controlled request-and-return loop.

A round repeats; it is not one long data transfer or a database synchronization.

1. PREPARE

Choose eligible sites, the approved starting model, code version, privacy profile, and round ID.

2. DISTRIBUTE

The controller offers each selected client a task carrying the model and approved instructions.

3. RUN LOCALLY

Each executor trains or evaluates beside local data and returns only its permitted contribution.

4. COMBINE + CHECK

The server aggregates results, creates a candidate, and sends it back for site-level evaluation.

Another round or release begins only after participant, privacy, quality, and governance gates pass.

APPENDIX · FEDAVG

FedAvg combines local updates into a new candidate.

It averages model changes: often weighted by local training size, not patient rows.

Same starting model

Every selected hospital begins the round from the same approved global-model version.

Train locally

Each site adjusts the model using only its approved local examples and training rules.

Weight + average

The server combines updates, commonly giving more weight to sites with more approved examples.

Evaluate candidate

The average becomes a candidate only; every site still checks quality and safety before release.

Simple example: 300 approved examples may receive 3× the weight of 100; real policies can differ / A6

APPENDIX · PRIVACY LAYERS

Federated learning still needs separate privacy layers.

Keeping raw data local reduces movement, but model updates and infrastructure can still leak or be attacked.

SECURE AGGREGATION

Reveals only a combined update when enough sites participate. It hides individual updates from the coordinator, but does not stop poisoning or final-model memorization.

DIFFERENTIAL PRIVACY

Clips contributions and adds calibrated noise. A privacy budget tracks protection across rounds; stronger privacy can reduce model usefulness.

IDENTITY + SECURE TRANSPORT

mTLS authenticates participants and encrypts the connection. Signatures and digests detect changed jobs or models; local authorization may still reject them.

Use layers together: no single control makes a federation private, secure, compliant, or clinically safe / A7

APPENDIX · RUN MODES

FLARE grows from laptop test to distributed production.

Each mode proves something different; a successful simulation is only the first piece of evidence.

SIMULATOR

Runs a lightweight multi-client experiment on one machine. Best for learning the API, testing algorithms, and checking basic job logic quickly.

PROOF OF CONCEPT

Runs separate local server and client processes. Tests task flow, configuration, operators, failures, and a closer version of the real runtime.

PRODUCTION

Runs provisioned participants across real machines or organizations with PKI, mTLS, authorization, deployment, monitoring, and recovery.

Progression: algorithm evidence → system evidence → real organizational and security evidence / A8

APPENDIX · OPERATIONAL TERMS

Five terms complete the federated-round picture.

These are the terms most likely to appear in a project review or professor's question.

Term	Plain meaning	Why it matters here
Seed / global model	The approved shared model before a round begins	Every eligible site must receive the intended version
Round	One distribute, local-work, aggregate, and evaluate loop	Needs one ID, participant list, versions, status, and stop reason
Local model update	A mathematical change computed by one site	Not raw rows, but still potentially sensitive or malicious
Candidate model	A newly combined model waiting for every check	It is not a release and must not silently replace the current model
Dropout / quorum	A missing site / the minimum completed sites required	Cancel safely when too few approved sites complete

Canary and rollback occur only after a candidate passes the release gates / A10

APPENDIX TAKEAWAY

You only need five ideas to explain the design.

FLARE runs the federation; Dagents governs how it becomes a controlled healthcare workflow.

- Raw data stays local, but model updates still need protection and review.
- A controller assigns tasks; client executors or client code run them locally.
- Repeated rounds create candidate models—not automatic clinical releases.
- Secure aggregation, differential privacy, mTLS, signatures, and authorization solve different risks.
- Move from simulator to POC to production only when each stage supplies the required evidence.